

Recap for Midterm

Midterm will be 35 points (35%)

- Short answer 15 Q (1 point each)
 - 2 Programming questions (40)
 - ↳ Easy: fill in the blank (1 blank = 1 point)
 - ↳ medium: on your own (20 lines)
line = 0.5 point
-

Input / Output / Processing

Variables: a place to store data

var_name = value
↑
assignment

use/call variable → replace it with the value

Variable types / data types

<u>String</u>	<u>integer</u>	<u>float</u>	<u>boolean</u>
text "Hello" "Hi!"	number without decimal 1, 2, 3, -3	real number 0.12, 1.23, -1.1	True, False ↑ condition
ex ^{var} <u>hello</u> = ^{string} <u>"hi"</u> print("hello") → hello			

Math Operators

$+$, $-$, $*$, $/$, $//$, $\div$, $**$

$18 // 4 = 4$ (100% box)
 $18 \div 4 = 4.5$
 $18 \times 4 = 72$
 $18 \times .4 = 7.2$

$18 \div 4 = 4$ (4)
 $18 \div 4 = 4.5$ (4.5)
 $18 \div 4 = 4.5$ (4.5)

Math Operator Order

1. $()$
2. $**$
3. $*, /, //, \div$
4. $+, -$

String Operators

$.$, $+$, $*$, $/$, $//$, $\div$, $**$

+ : concatenate two strings together

"Hello" + "ABC" → "HelloABC"

"1" + "2" + "3" → "123"

Output: print() function

things you want to print

Arguments

- end = "str" → replace new line with "str"
100% because the answer will always be one line.
Answer select

print('A') A
print('B') → B
print('C') C

print('A', end=':')
print('B', end='-')
print('C', end=',')

A:B-C

1	4
2	5
3	6

⊗ forget end='...' → -25% = 0.25

- sep = "str" → 20% replace space with "str" when printing multiple things

f_string : formatted-string

Syntax: f" {code : .nt} "

Syntax:

f" {code : .nt} "

execute the code and place the out come here.
↑ Format decimal
n = decimal place

x = f" Hello {10 * 2 : .2f} Hi "

print(x) → Hello 20.00 Hi

n = input(x)

Input:

input() function

Optional text to show to the user

* input returns a string
↳ save to a variable

if you want a number, you need to convert:

↳ int(str) : str to int

↳ float(str) : str to float

has to be a valid number

int("123") → 123

int("A1") ✗

* be careful with int, float.

in ' ' n is integer

* became tuple with int, float.

Ex → $N = \text{int}(\text{input})$ | N is integer
 $X = \text{float}(\text{input})$ | X is a real number (-50%)
↓
name = input("Enter your name: ")

Named Constants

↳ a variable in all UPPER CASE that store a constant.

↳ The programming questions will ask you to use it.

Ex. RATE = 20

(0.5 point) ✓

↳ What can be constants?

↳ Price of goods that doesn't change.

↳ Threshold (min, max) for input validation

↳ any numbers on the question

~~~~~ 10 ~~~~~ 70  
~~~~~ 1 ~~~~~ 10 ~~~~~

Decision Structures

and also

if-else

If

If condition:
[statement]
[statement]

↑
space to indicate that
statements are inside
the if block.

If-else

If condition:
[statement]
[statement]
else:
[statement]
[statement]

If-elif-else

if condition:
[statement]
elif condition:
[statement]
elif condition:
[statement]
else:
[statement]

If

score = 95

if score >= 80:
print('A')
if score >= 70:
print('B')

vs.

if-elif-else

score = 95

if score >= 80:
print('A')
elif score >= 70:
print('B')

Comparison Operators → True, False

>, <, >=, <=, ==, !=
equal?, not equal?

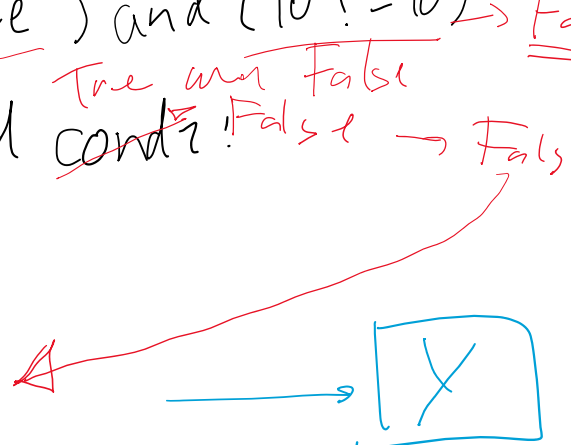
Boolean Operators: and, or, not

True or True → True

Boolean Operations . and / or / not

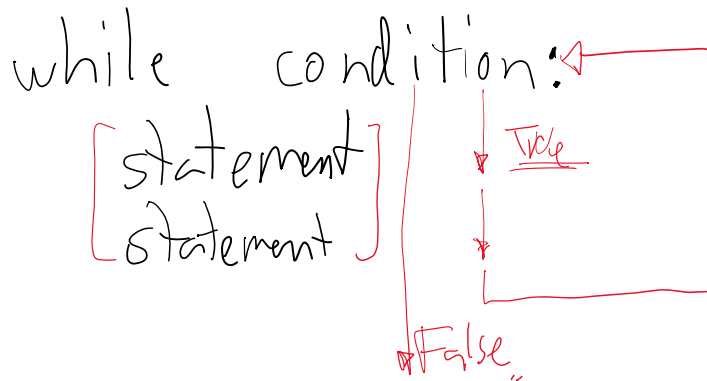
EX

Cond1 = (10 == 5) or (5 > 3) → True
Cond2 = (True) and (10 != 10) → False
if Cond1 and Cond2 → False
 print('X')
else:
 print('X')



Repetition Structures

while loop: loop while the condition is true.



Input Validation

(100%)
* check age and keep asking if below 0.

age = int(input())

while age < 0:

while True:

 age = int(input())

 in

```

while age < 0:
    print("age must be >= 0")
    age = int(input())
print(age)

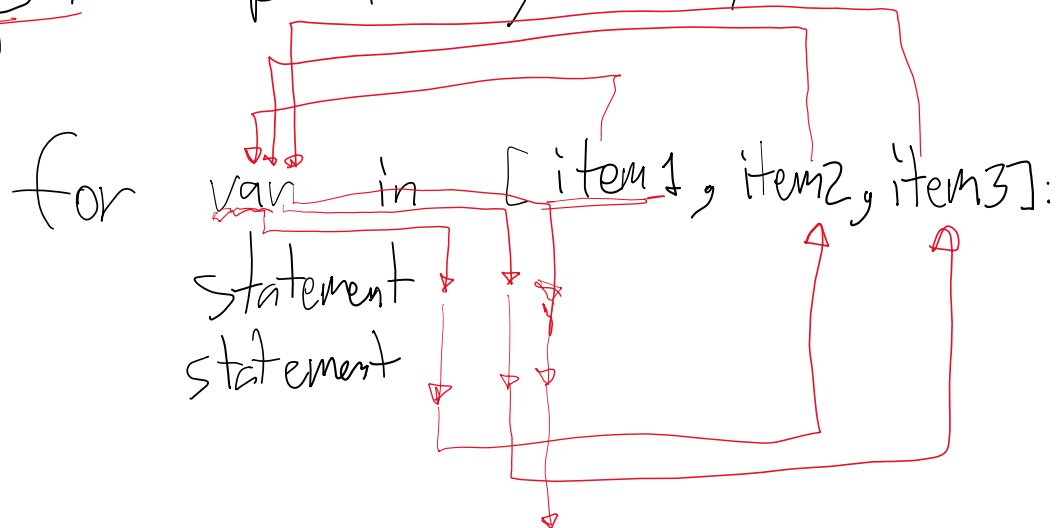
```

```

age = int(input())
if age >= 0:
    break ✓
print("age must be >= 0")
print(age)

```

for loop: loop through a sequence of items



range() function: create a sequence of numbers

↳ mainly use it with for loop

1) range(start, stop, step)

ex: range(1, 5, 2) → [1, 3]

↳ range(7, 1, -3) → [7, 4]

↳ range(1, 5, -3) → ? → []

↳ `range(1, 5, -3)` → ? → `[]`

↳ `range(1, 1, 1)` → `[]`

2) `range(start, stop)` → # step = 1

↳ `range(1, 5)` → `[1, 2, 3, 4]`

3) `range(stop)` → # start = 0, step = 1

↳ `range(5)` → `[0, 1, 2, 3, 4]`

for x in `range(6, 2, -2)`:
 `print(x)`

↳ 5 items [6, 4]
↳ Output: 6
 4

Short-hand Notation

`x = x + 1` → `x += 1`

`total = total + n` → `total += n`

`y = y * 2` → `y *= 2`

Problem

- Ask user for n items
- User will enter n first
- Then, you loop n times

- Then you loop n times
asking for n items

```
n = int(input("Enter n: "))  
total = 0  
for i in range(n)
```

$[0, 1, 2, \dots, n-1]$
n items $\rightarrow$ n loops

```
    cost = float(input())
```

```
    total += cost
```

outside
 $\rightarrow$
 $\rightarrow$

```
avg = total / n
```

```
print(f"Avg. cost is {avg:.2f}")
```

* logics : - calculate total
- count something based on condition
- loop n times to get n inputs

List (part 1)

List is a collection of items.

Syntax: $[\text{item1}, \text{item2}, \text{item3}]$

* - forget [] $\leftarrow$ $\boxed{\begin{matrix} -25\% \\ -0.25 \end{matrix}}$

.. ✓ [1 > 2]

ex. $x = [1, 2, 3]$

$\text{print}(x) \rightarrow 1, 2, 3 = -0.25$

target LJ $\leftarrow 1 \frac{0.1}{-0.25}$

Indexing: access the item based on index

- index starts at 0.

- negative index ends at -1.

my_list = ^{0 1 2 3 4}
[1, 2, 3, 4, 5]

^{-5 -4 -3 -2 -1}
 $\text{print}(\text{my_list}[0]) \rightarrow$

1

$\text{print}(\text{my_list}[-2]) \rightarrow$

4

Update: update based on index

Syntax: list[index] = new_value

Ex

my_list = ^{0 1 2 3 4}
[10, 20, 30, 40, 50]

my_list[0] = 5

my_list[2] = $\text{my_list}[0] + \text{my_list}[-1]$

$\text{print}(\text{my_list}) \rightarrow [5, 20, 55, 40, 50]$

List Operators

List Operators

- $lst + lst$: concatenate two lists together
 $[1, 2] + [3, 4] \rightarrow [1, 2, 3, 4]$

- $lst * n$: concatenate lst with itself n times
 $[1, 2] * 3 \rightarrow [1, 2, 1, 2, 1, 2]$

len() function $\rightarrow$ return the numbers of items

⊗ loop through list and index of list

my_list = [10, 20, 30]

range(3) $\rightarrow$ [0, 1, 2]

for i in range(len(my_list)):

print(i, my_list[i])

| Output! | |
|---------|----|
| 0 | 10 |
| 1 | 20 |
| 2 | 30 |

Nested loop

~ hard Question
(~1-3 nested loops)

total = 0
v = 1

while #1
 $x < 5 \rightarrow 1 < 5$ 0

total = 0

x = 1

while x < 5:

for i in range(x):

if i % 2 == 0:

total += 2

total += x

x *= 2

print(total)

+2 +1 +2 +2 }
+2 +2 +4 } 15

x < 5 → 1 < 5

for i in range(1)

for # 1 0 % 2 == 0

if i % 2 == 0:

total += 2

total += 1

x * 2 = 1 * 2 = 2

while # 2

for i in range(2)

total += 2

total += 2

x * 2 = 2 * 2 = 4

while # 3

for i in range(4)

total += 2 2 times

total += 4

x * 2 → 4 * 2 = 8